

UNIVERSIDAD CATÓLICA DE TEMUCO

Informe técnico del analizador léxico para un subconjunto de Prolog

Teoría de la Computación

Dato	Información
Código del curso	INFO1148
Asignatura	Teoría de la Computación
Integrantes	Jose Jimenez Saavedra — jose.jimenez2023@alu.uct.cl Franco Oyarzo Calisto — foyarzo2023@alu.uct.cl Demian Quezada Pasten — dquezada2023@alu.uct.cl
Profesor	Prof. M. Lévano
Jefe de grupo	Jose Jimenez Saavedra
Periodo académico	Segundo semestre de 2026

Temuco, septiembre de 2026

Resumen

El trabajo presenta el diseño, la implementación y la validación de un analizador léxico para un subconjunto de Prolog. La solución, desarrollada en Python, recorre la fuente de izquierda a derecha y reconoce átomos, variables, números, cadenas, operadores y delimitadores. Los espacios y comentarios se descartan sin perder el seguimiento de línea y columna. Para resolver lexemas con prefijos compartidos se aplica máxima coincidencia; además, los operadores alfabéticos `is` y `mod` se clasifican después de consumir el identificador completo. Cada token conserva su tipo, lexema, posición y, cuando corresponde, un índice en la tabla de lexemas. El signo de los números se trata como operador independiente, de modo que no se confunde el reconocimiento léxico con la interpretación sintáctica. La validación incluye 75 pruebas automatizadas, un programa correcto y otro con errores recuperables. Las pruebas finalizaron sin fallos; el archivo válido produjo 94 tokens y ningún error, mientras que el archivo inválido produjo 30 tokens y 6 diagnósticos. El alcance no incluye análisis sintáctico, semántica ni compatibilidad completa con ISO Prolog.

Palabras clave: análisis léxico, Prolog, autómatas finito, máxima coincidencia, tabla de lexemas.

1 Introducción

El análisis léxico transforma una secuencia de caracteres en unidades que pueden ser tratadas por las etapas posteriores de un compilador o intérprete. En este trabajo, el problema consiste en recorrer archivos escritos en un subconjunto de Prolog, distinguir las categorías definidas y asociar a cada token su posición inicial. El recorrido también debe descartar comentarios y espacios sin alterar el cálculo de línea y columna, además de informar errores léxicos sin detenerse ante el primer carácter inválido.

La dificultad no se reduce a comparar caracteres con una lista. Varias categorías comparten prefijos: `\==` comienza como `\=`, `=..` comienza como `=` y el átomo `island` contiene el prefijo `is`. Si el analizador aceptara el primer patrón disponible, fragmentaría lexemas que pertenecen a una sola unidad. Por esa razón, la solución conserva la aceptación provisional más larga y aplica una prioridad estable cuando dos reglas pueden reconocer la misma entrada, criterio habitual en la construcción de analizadores léxicos [1].

El informe define primero el subconjunto admitido y sus expresiones regulares. Luego relaciona esas definiciones con AFD para identificadores, números y operadores, e incluye una determinización y una minimización representativas. Las secciones posteriores describen la arquitectura, la tabla de lexemas, el tratamiento de errores y los resultados del corpus automatizado. De este modo, la especificación formal y el comportamiento del programa pueden contrastarse sobre los mismos casos.

2 Objetivos

2.1 Objetivo general

Desarrollar y validar un analizador léxico para un subconjunto definido de Prolog, relacionando expresiones regulares y autómatas finitos con una implementación en Python capaz de generar tokens posicionados y diagnósticos recuperables.

2.2 Objetivos específicos

Los objetivos específicos consisten en definir las categorías admitidas mediante patrones, ejemplos, atributos y reglas de prioridad; modelar categorías representativas con autómatas finitos y documentar una conversión de AFN a AFD junto con su minimización; implementar el recorrido de la fuente, la tabla de lexemas y

los diagnósticos de error; y comprobar el comportamiento del analizador mediante pruebas automatizadas y dos archivos completos de entrada.

3 Alcance del analizador

La especificación restringe los identificadores y dígitos a caracteres ASCII. Esta decisión mantiene explícitas las clases de transición del autómata y acota la implementación al subconjunto pedido, aunque SWI-Prolog admite identificadores Unicode más amplios [4]. Los literales citados sí pueden contener otros caracteres, siempre que no introduzcan CR o LF ni una secuencia de escape fuera de la convención adoptada. Los separadores admitidos son espacio, tabulación, CR, LF, salto de página y tabulación vertical; CRLF cuenta como un solo salto de línea. Las posiciones comienzan en 1 y cada tabulación ocupa una columna lógica, independientemente de su ancho visual en el editor.

El analizador produce tokens y errores léxicos, pero no determina si esos tokens forman una cláusula correcta. Tampoco interpreta precedencias, realiza unificación, administra ámbitos ni asigna significado a los literales. La Tabla 1 resume esa frontera para evitar que los resultados se presenten como una implementación completa de Prolog.

Tabla 1: Decisiones de alcance del analizador.

Aspecto	Incluido	Excluido
Identificadores	Átomos iniciados en minúscula; variables iniciadas en mayúscula o guion bajo; <code>_</code> como anónima.	Unicode fuera de literales citados.
Números	Enteros, decimales con dígitos a ambos lados del punto y exponentes opcionales.	Bases no decimales, racionales, infinidades y NaN.
Texto	Átomos con comillas simples y cadenas dobles; escapes <code>\n</code> , <code>\t</code> , <code>\r</code> , <code>\\</code> , <code>\'</code> y <code>\"</code> .	Salto de línea sin escapar y otros escapes.
Operadores	Cláusulas, consultas, unificación, comparación, aritmética y control señalados en la tarea.	Operadores definidos por el usuario.
Comentarios	<code>%</code> hasta fin de línea y <code>/* ... */</code> .	Comentarios de bloque anidados.
Procesamiento	Tokenización, posiciones, tabla de lexemas y recuperación léxica.	Sintaxis, precedencia, unificación, ámbitos y semántica.

4 Especificación léxica

Se define $L = [a-z]$, $U = [A-Z]$, $D = [0-9]$ y $C = L \cup U \cup D \cup \{_ \}$. El alfabeto de entrada Σ contiene los caracteres Unicode de la fuente, pero los identificadores, dígitos y espacios admitidos se restringen a las clases ASCII indicadas. Los caracteres Unicode adicionales pueden aparecer dentro de citas y comentarios. LF, CRLF y CR se cuentan como un único salto de línea; una tabulación ocupa una columna. Las posiciones comienzan en uno y cuentan caracteres, no bytes ni anchura visual.

Las expresiones de la Tabla 2 usan la notación de **re** de Python: `|` denota alternativas, los corchetes definen clases y los grupos `(?:...)` no capturan. Los patrones se aplican desde la posición actual mediante `match`; no se busca una coincidencia posterior que omita caracteres. En Python las alternativas se intentan de izquierda a derecha, de modo que el orden de operadores largos antes de sus prefijos forma parte de la implementación de máxima coincidencia [3]. Las citas y los comentarios de bloque se recorren explícitamente para conservar

el fragmento de los errores de cierre.

Tabla 2: Definición formal del subconjunto.

Token	Descripción	Expresión regular conceptual
ATOMO	Identificador iniciado en minúscula; se reclasifican is y mod .	<code>[a-z][A-Za-z0-9_]*</code>
ATOMO_CITADO	Texto entre comillas simples con escapes admitidos.	<code>'(?:\\[ntr\\'"] \\'\\\\r\\\\n\\')*'</code>
VARIABLE	Identificador iniciado en mayúscula o guion bajo, excepto el lexema aislado <code>_</code> .	<code>(?:[A-Z][A-Za-z0-9_]* _[A-Za-z0-9_]+)</code>
VARIABLE_ANONIMA	Guion bajo aislado.	<code>-</code>
ENTERO	Secuencia decimal sin signo.	<code>[0-9]+</code>
REAL	Decimal o exponente; el signo solo se admite en el exponente.	<code>(?:[0-9]+\\. [0-9]+(?:[eE][+-]?[0-9]+)? [0-9]+[eE][+-]?[0-9]+)</code>
CADENA	Texto entre comillas dobles con escapes admitidos.	<code>"(?:\\[ntr\\'"] \\'\\\\r\\\\n\\')*"</code>
OPERADOR_CLAUSULA	Operadores de regla y gramática de cláusulas definidas.	<code>--> :-</code>
OPERADOR_CONSULTA	Inicio de consulta.	<code>\\?-</code>
OPERADOR_COMPARACION	Unificación, identidad, desigualdad y comparación.	<code>\\== =\\.\\. \\ = == < > = < ></code>
OPERADOR_ARITMETICO	Operadores simbólicos y alfabéticos admitidos.	<code>// ** [+\\-*/] is mod</code>
OPERADOR_CONTROL	Negación, corte y disyunción.	<code>\\+ ! ;</code>
DELIMITADORES	Familia de tokens detallada en la Tabla 3.	<code>[()\\[\\]\\{\\} ,.]</code>
COMENTARIO de línea	Desde % hasta antes de CR, LF o fin de archivo; se descarta.	<code>%[^\\r\\\\n]*</code>
COMENTARIO de bloque	Hasta el primer cierre; sin anidamiento; se descarta.	<code>/* [^*]**+(?: [^/*] [^*]**+)* /</code>
ESPACIO	Separadores descartados con actualización de posición.	<code>[\\t\\r\\\\n\\f\\v]+</code>

Los operadores cubren los lexemas obligatorios de la tarea y la extensión opcional `-->`. Prolog permite declarar operadores y asociarles precedencia y tipo, pero esa información pertenece al análisis sintáctico y queda fuera de este lexer [5]. Se reconoce primero el identificador completo: solo los lexemas exactos **is** y **mod** se reclasifican como operadores; **island**, **modulo** e **is2** son átomos. Entre números válidos, **REAL** tiene prioridad sobre **ENTERO**. El signo inicial siempre es un operador: `-25` produce dos tokens, mientras que `2e-3` produce un **REAL**.

Ambas clases de citas permiten contenido vacío y los escapes `\\n`, `\\t`, `\\r`, `\\`, `\\'` y `\\"`. Se conserva el lexema exacto, incluidas comillas y barras, sin evaluar los escapes. Un CR o LF literal interrumpe una cita sin

cierre. Los comentarios no se reconocen dentro de citas; las comillas no tienen significado especial dentro de comentarios. La expresión de bloque impide consumir un cierre intermedio, por lo que `/* a */ b /* c */` contiene dos comentarios separados por el átomo `b`.

La aceptación numérica incorpora una regla de frontera: una letra ASCII, guion bajo o un segundo punto seguido de dígito ASCII inmediatamente después del número origina un diagnóstico de número mal formado. Así, `12abc` y `3.14.15` no se fragmentan en tokens válidos. Un punto sin dígito posterior queda disponible como PUNTO: `12.` produce ENTERO y PUNTO. Esta comprobación y la recuperación de errores complementan los patrones de números válidos; no analizan la gramática de cláusulas.

Tabla 3: Nombres concretos de los tokens delimitadores. Cada patrón es un carácter literal.

Token	Lexema	Token	Lexema
PARENTESIS_IZQ	(	PARENTESIS_DER	)
CORCHETE_IZQ	[	CORCHETE_DER	]
LLAVE_IZQ	{	LLAVE_DER	}
BARRA_VERTICAL		COMA	,
PUNTO	.		

Los ejemplos inválidos de la Tabla 4 no pertenecen a la categoría de su fila; algunos sí son válidos como otra categoría. Por ejemplo, `Padre` es una variable y `__` también. Los comentarios y espacios no emiten tokens ni atributos; los índices de átomos, variables y literales comienzan en cero dentro de sus respectivas tablas.

Tabla 4: Ejemplos y atributos asociados.

Token	Ejemplos válidos	Ejemplos inválidos	Atributo
ATOMO	<code>padre, persona_1</code>	<code>Padre, 1persona</code>	Índice en átomos
ATOMO_ CITADO	<code>'Juan Pérez', ':-'</code>	<code>'sin cierre</code>	Índice en átomos
VARIABLE	<code>X, Persona, _Tmp</code>	<code>x, 2X</code>	Índice en variables
VARIABLE_ ANONIMA	<code>-</code>	<code>__ como anónima</code>	Sin índice
ENTERO	<code>25, 0</code>	<code>12abc, 2e</code>	Índice en literales
REAL	<code>3.14, 2e-3</code>	<code>3.14.15, 1e+</code>	Índice en literales
CADENA	<code>"hola", "a\n b"</code>	<code>"sin cierre, "x\q"</code>	Índice en literales
OPERADOR_ CLAUSULA	<code>:-, --></code>	<code>:, ?-</code>	Sin índice
OPERADOR_ CONSULTA	<code>?-</code>	<code>?, :-</code>	Sin índice
OPERADOR_ COMPARACION	<code>\==, =.., =<</code>	<code>is, @</code>	Sin índice
OPERADOR_ ARITMETICO	<code>//, **, is</code>	<code>island, =</code>	Sin índice
OPERADOR_ CONTROL	<code>\+, !, ;</code>	<code>+, ?</code>	Sin índice
DELIMITADORES	<code>() [] { } , .</code>	<code>@, #</code>	Sin índice
COMENTARIO	<code>% nota, /* nota */</code>	<code>/* sin cierre</code>	Se descarta
ESPACIO	<code>Espacio, TAB, CR, LF, FF, VT</code>	<code>Espacio Unicode U+00A0</code>	Se descarta

5 Diseño de los autómatas

Los autómatas de las Figuras 1, 2 y 3 representan las categorías con mayor posibilidad de superposición. La flecha de entrada identifica el estado inicial q_0 y la doble circunferencia identifica los estados aceptadores, con la categoría indicada en su interior. Las transiciones omitidas llevan a un estado muerto implícito. Al tokenizar, se conserva la última aceptación y se devuelve el prefijo aceptado más largo; el carácter que inicia el siguiente token queda sin consumir. La validación de fronteras numéricas puede sustituir ese prefijo por un diagnóstico, según la especificación anterior.

AFD para identificadores

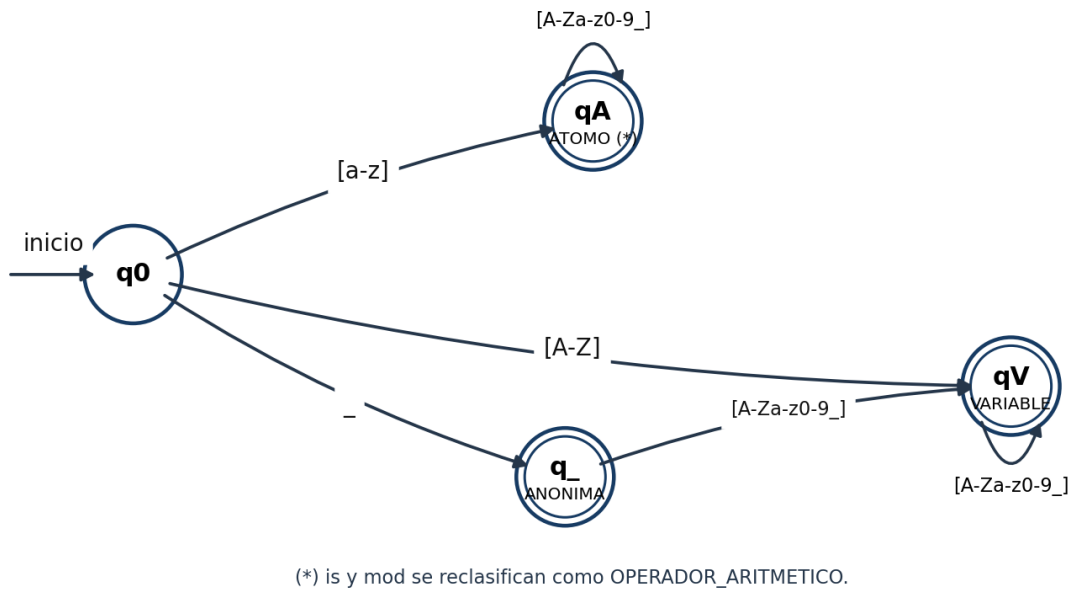


Figura 1: AFD para identificadores. El estado q_A requiere reclasificar `is` y `mod`.

AFD para enteros y reales sin signo

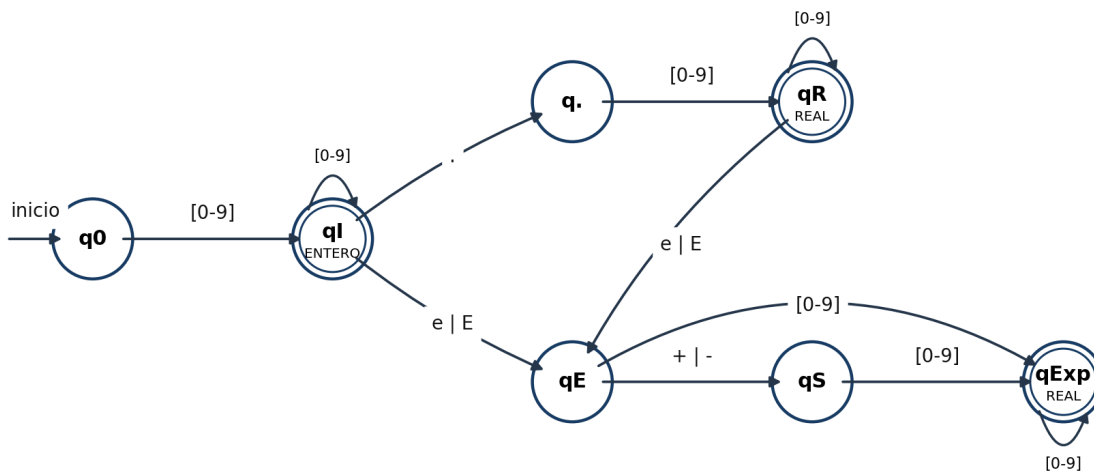


Figura 2: AFD para enteros y reales sin signo léxico.

Trie de operadores y máxima coincidencia

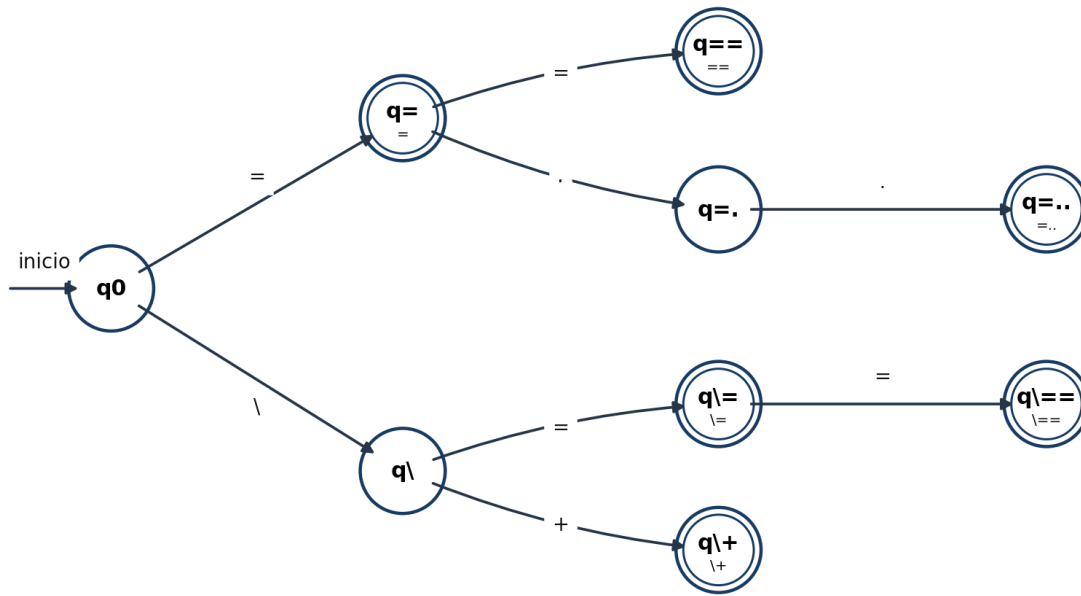


Figura 3: Fragmento del trie de operadores con prefijos compartidos.

El estado q_{-} acepta únicamente la variable anónima. Si se lee otro carácter de C , se pasa a q_V y el lexema completo es una variable con nombre. En el autómata numérico, q_I , q_R y q_{Exp} aceptan ENTERO, REAL y REAL, respectivamente; q_{-} , q_E y q_S no aceptan. Por ejemplo, **2e-3** recorre $q_0, q_I, q_E, q_S, q_{Exp}$. Para **12.**, la última aceptación es q_I : el punto queda como delimitador. Para **1e+**, la frontera detecta un exponente incompleto y produce un error, en lugar de emitir el entero 1.

El trie mostrado es un fragmento: = y \= ya aceptan, pero pueden prolongarse hasta ==, =. y \==. En el lexer, la tupla completa **SYMBOL_OPERATORS**, ordenada por longitud descendente, representa las demás ramas. Un delimitador requiere una transición desde el inicio a su estado aceptador; espacios y comentarios terminados alcanzan estados de descarte.

5.1 Autómatas para citas y comentarios

Un mismo AFD sirve para ATOMO_CITADO y CADENA parametrizando la comilla c (simple o doble). Sea $E = \{n, t, r, \backslash, \text{comilla simple}, \text{comilla doble}\}$ el conjunto permitido después de una barra. La Tabla 5 especifica todas las transiciones que no van al estado muerto. Q_0 es inicial y Q_F es el único aceptador: emite el tipo determinado por c . Las clases de cada fila son disjuntas; en Q_I la comilla opuesta es un carácter ordinario.

Tabla 5: AFD parametrizado para átomos citados y cadenas.

Origen	Entrada	Destino	Interpretación
Q_0	c	Q_I	Apertura
Q_I	$\Sigma \setminus \{c, \backslash, CR, LF\}$	Q_I	Contenido ordinario
Q_I	$\backslash$	Q_E	Inicio de escape
Q_E	E	Q_I	Escape admitido
Q_I	c	Q_F	Cierre y aceptación

CR, LF o fin de archivo antes de Q_F generan un diagnóstico de cita sin cierre. Un escape fuera de E impide aceptar el literal; la implementación conserva la primera secuencia inválida y continúa hasta su cierre o fin de línea para recuperar el análisis. Este recorrido de recuperación no convierte el literal erróneo en un token aceptado.

Para comentarios de línea, el estado inicial L_0 pasa con % a L_1 , que es aceptador y repite con $\Sigma \setminus \{\text{CR}, \text{LF}\}$. Al encontrar un salto o fin de archivo se descarta el comentario. El salto queda para el reconocedor de espacios. Para comentarios de bloque, la Tabla 6 reconoce el primer cierre $*/$; B_0 es inicial y B_F es el único aceptador. Un fin de archivo en B_2 o B_3 produce un error de comentario sin cierre. La entrada $/*$ activa este reconocedor antes que el operador $/$.

Tabla 6: Transiciones no muertas del AFD de comentarios de bloque.

Origen	Entrada	Destino	Interpretación
B_0	$/$	B_1	Primer carácter
B_1	$*$	B_2	Apertura completa
B_2	$\Sigma \setminus \{*\}$	B_2	Contenido
B_2	$*$	B_3	Posible cierre
B_3	$*$	B_3	Mantiene posible cierre
B_3	$\Sigma \setminus \{*, /\}$	B_2	Continúa contenido
B_3	$/$	B_F	Cierre y descarte

5.2 Determinización representativa

Para mostrar la construcción de subconjuntos se utiliza el lenguaje LC^* de identificadores iniciados en minúscula, antes de reclasificar las palabras **is** y **mod**. En este ejemplo, la etiqueta ATOMO designa ese patrón estructural. Se parte de un AFN deliberadamente no mínimo que divide la primera letra en dos ramas equivalentes. El estado q_0 posee transiciones ε hacia p_0 y r_0 ; desde p_0 , una letra de $[a-m]$ lleva a p_1 , mientras que desde r_0 una letra de $[n-z]$ lleva a r_1 . Los estados p_1 y r_1 son aceptadores y repiten con cualquier carácter de C . La transformación aplica la construcción de subconjuntos y luego refina particiones para obtener estados equivalentes [2].

Formalmente, $Q = \{q_0, p_0, r_0, p_1, r_1\}$, el inicial es q_0 y $F = \{p_1, r_1\}$. Las clases de entrada son $[a-m]$, $[n-z]$, $U \cup D \cup \{_ \}$ y $\Sigma \setminus C$. Para esta última, todas las transiciones del AFN son vacías; se omite su columna para facilitar la lectura.

Tabla 7: Transiciones del AFN representativo.

Estado	ε	$[a-m]$	$[n-z]$	$U \cup D \cup \{_ \}$
q_0	$\{p_0, r_0\}$	$\emptyset$	$\emptyset$	$\emptyset$
p_0	$\emptyset$	$\{p_1\}$	$\emptyset$	$\emptyset$
r_0	$\emptyset$	$\emptyset$	$\{r_1\}$	$\emptyset$
p_1	$\emptyset$	$\{p_1\}$	$\{p_1\}$	$\{p_1\}$
r_1	$\emptyset$	$\{r_1\}$	$\{r_1\}$	$\{r_1\}$

Para un subconjunto S y un carácter a se calcula

$$\delta_D(S, a) = \text{cl}_\varepsilon \left(\bigcup_{q \in S} \delta_N(q, a) \right).$$

La clausura- ε inicial es $D_0 = \{q_0, p_0, r_0\}$. Al mover ese conjunto con $[a-m]$ se obtiene $D_1 = \{p_1\}$; con $[n-z]$ se obtiene $D_2 = \{r_1\}$. Cualquier clase sin transición conduce a $D_\perp = \emptyset$. Como p_1 y r_1 repiten con C , no aparecen nuevos subconjuntos. Un subconjunto acepta si intersecta F , por lo que solo D_1 y D_2 son aceptadores. Los cuatro estados son alcanzables: con ε , **a**, **n** y **0**, respectivamente.

Tabla 8: Tabla de transición obtenida mediante construcción de subconjuntos.

Estado AFD	$[a-m]$	$[n-z]$	$U \cup D \cup \{_\}$	Aceptación
$D_0 = \{q_0, p_0, r_0\}$	D_1	D_2	$D_\perp$	No
$D_1 = \{p_1\}$	D_1	D_1	D_1	ATOMO
$D_2 = \{r_1\}$	D_2	D_2	D_2	ATOMO
$D_\perp = \emptyset$	$D_\perp$	$D_\perp$	$D_\perp$	No

Para todo estado de la Tabla 8, una entrada de $\Sigma \setminus C$ conduce a $D_\perp$. Por ejemplo, **padre_1** sigue $D_0, D_2, D_2, D_2, D_2, D_2, D_2, D_2$ y acepta; **Padre** pasa inmediatamente al estado muerto. No se confunde rechazo como átomo con error global: el despachador reconoce **Padre** mediante el AFD de variables.

5.3 Minimización

La partición inicial es $P_0 = \{\{D_1, D_2\}, \{D_0, D_\perp\}\}$: el primer bloque contiene los estados que aceptan ATOMO y el segundo, los que no aceptan. El bloque no aceptador se divide porque desde D_0 una letra minúscula alcanza aceptación, mientras que desde $D_\perp$ toda entrada permanece en el estado muerto. En cambio, D_1 y D_2 no se separan: ambos aceptan exactamente los sufijos de C^* y cualquier carácter fuera de C lleva al mismo estado muerto. La partición estable es $P_1 = \{\{D_1, D_2\}, \{D_0\}, \{D_\perp\}\}$, por lo que D_1 y D_2 se fusionan en la clase de equivalencia $D_A = [D_1] = [D_2]$.

Los tres estados restantes son distinguibles: el sufijo vacío ε separa D_A de ambos estados no aceptadores, y el sufijo **a** distingue D_0 de $D_\perp$. Como todos son alcanzables, el AFD completo de tres estados es mínimo. En los diagramas parciales se omite el estado muerto y quedan dos estados visibles para este patrón. Esta minimización no corresponde al lexer entero: un AFD que emita varias categorías debe conservar las etiquetas de token al formar su partición inicial.

Tabla 9: AFD mínimo para el ejemplo representativo.

Estado mínimo	$[a-m]$	$[n-z]$	$U \cup D \cup \{_\}$	Aceptación
D_0	D_A	D_A	$D_\perp$	No
$D_A = [D_1] = [D_2]$	D_A	D_A	D_A	ATOMO
$D_\perp$	$D_\perp$	$D_\perp$	$D_\perp$	No

Las entradas de $\Sigma \setminus C$ llevan a $D_\perp$ también en la Tabla 9. La reclasificación de **is** y **mod** se mantiene después del reconocimiento: fusionar los estados estructurales no elimina la consulta del lexema completo.

5.4 Estrategia integrada y correspondencia con el código

Trazas de aceptación y fronteras

Las siguientes trazas separan la aceptación del autómata de la decisión del lexer sobre la frontera del lexema. Un estado no aceptador intermedio no implica que toda la entrada sea errónea: puede existir una aceptación anterior. A su vez, la regla numérica de frontera puede impedir emitir ese prefijo si ocultaría un número mal formado.

Tabla 10: Trazas representativas y resultado observable.

Entrada	Recorrido relevante	Resultado del lexer
12.	$q_0, q_I, q_I, q_.$ La última aceptación está antes del punto.	ENTERO 12, PUNTO ..
2e-3	$q_0, q_I, q_E, q_S, q_{Exp}$. El signo pertenece al exponente.	Un REAL con lexema 2e-3.
1e+	q_0, q_I, q_E, q_S . El estado final no acepta y el sufijo inicia con e.	Un error por 1e+; no se emite ENTERO.
3.14.15	Se alcanza q_R en 3.14; la frontera siguiente es punto más dígito.	Un error por todo 3.14.15.
_X	$q_0, q_., q_V$. La aceptación de variable prolonga la anónima.	VARIABLE con índice; no se emite _ separado.
is2	q_0, q_A, q_A, q_A . Se consulta el lexema completo.	ATOMO, porque no coincide exactamente con is.

Los casos numéricos se verifican en `tests/test_lexer.py` y `tests/test_lexer_regressions.py`. Estos archivos también contrastan operadores adyacentes sin espacios, prefijos incompletos y la prioridad de las citas sobre los marcadores de comentario. El caso `)][(,;. se acepta léxicamente aunque no forme una cláusula: comprobar el orden de los delimitadores sería análisis sintáctico y queda fuera del enunciado.`

Despacho de reconocedores

El programa no materializa un AFD único en memoria. Utiliza una estrategia equivalente: un despachador selecciona la categoría a partir del carácter inicial. Primero descarta espacios y comentarios; después reconoce citas, números e identificadores; finalmente intenta operadores, delimitadores y el diagnóstico de carácter no admitido. Las expresiones IDENTIFIER, REAL e INTEGER describen los lenguajes de sus AFD, y SYMBOL_OPERATORS codifica las hojas del trie. Esto establece equivalencia de reconocimiento; no afirma que el motor `re` de Python ejecute internamente un AFD.

Las superposiciones se resuelven de manera explícita: `/*` tiene prioridad sobre `/`; una cita consume su interior antes de reconocer operadores; un identificador se completa antes de reclasificar palabras; y los operadores se prueban por longitud descendente. Todos los reconocedores aceptan un lexema, descartan un separador o producen un diagnóstico que avanza la lectura. El ciclo principal conserva una sola posición y termina cuando llega al final de la fuente. Así se relacionan los autómatas, las reglas de prioridad y la recuperación sin incorporar análisis sintáctico.

6 Reglas de prioridad y máxima coincidencia

Tabla 11: Resolución de superposiciones entre patrones.

Caso	Regla aplicada	Resultado
\== / \= / ==	Los operadores se prueban por longitud descendente. El lexer no confirma \= si existe otro signo igual consecutivo.	\== cuando la entrada es \==.
=.. / =	Tras reconocer = como aceptación provisional, se continúa por dos puntos si están disponibles.	=.. cuando la entrada es =..
:- / :	Solo :- pertenece al subconjunto. Un : aislado genera un diagnóstico.	:- o error en :.
_ / _Variable	Se reconoce primero el identificador completo. Solo el lexema exacto _ se clasifica como anónimo.	VARIABLE_ANONIMA o VARIABLE.
is / island	Se consume la palabra completa y después se consulta la tabla de operadores alfabéticos.	OPERADOR_ ARITMETICO o ATOMO.
-25	El signo queda fuera del patrón numérico y se emite como operador independiente.	- seguido de ENTERO 25.

La implementación aplica estas reglas dentro de un orden de decisión único. Primero reconoce espacios y comentarios, porque no deben producir tokens. Después procesa los literales citados, los números y los identificadores. Los operadores simbólicos se comparan por longitud descendente y los delimitadores se dejan para el final. Si ninguna categoría acepta el carácter actual, se registra un error y se avanza una posición.

Esta secuencia evita dos problemas concretos. Un comentario de bloque debe detectarse antes del operador división, y una palabra completa debe reconocerse antes de decidir si corresponde a `is`, `mod` o a un átomo ordinario. La prioridad, por tanto, no reemplaza la máxima coincidencia; ambas reglas trabajan juntas.

7 Diseño e implementación

7.1 Arquitectura de la solución

Tabla 12: Componentes de la implementación.

Archivo	Componente	Responsabilidad
src/lexer.py	Token y LexError	Representan salidas inmutables con tipo, lexema, posición y atributo opcional.
src/lexer.py	SymbolTable	Interna átomos, variables y literales; reutiliza el índice de un lexema repetido.
src/lexer.py	Lexer.scan	Recorre la fuente una vez, aplica prioridad, actualiza posiciones y recupera errores.
src/cli.py	Interfaz CLI	Lee UTF-8 con BOM opcional y emite texto o JSON en UTF-8; retorna 0, 1 o 2 según el resultado.
tests/	Pruebas	Comprueba categorías, posiciones, recuperación, interfaz y corpus completo.
corpus/	Entradas completas	Contiene un programa sin errores y otro con varios errores recuperables.
scripts/validate.py	Evidencia reproducible	Ejecuta la suite y el corpus; actualiza resultados JSON y cifras del informe.

La interfaz carga el archivo como texto UTF-8, acepta un BOM inicial opcional y lo entrega a `Lexer.scan` sin alterar sus saltos de línea. El recorrido mantiene el índice absoluto, la línea y la columna. Antes de consumir un lexema guarda su posición inicial; después actualiza el contador con el texto realmente leído. Los identificadores y números se reconocen mediante patrones compilados, mientras que los literales citados y comentarios de bloque se procesan manualmente para distinguir cierre, salto de línea y escape. Los operadores simbólicos se consultan en una secuencia ordenada desde el lexema más largo. Los códigos de salida son 0 si no hay errores léxicos, 1 si se detectaron errores recuperables o hasta EOF, y 2 ante un archivo ilegible, UTF-8 inválido o argumentos incorrectos.

El resultado agrupa la lista de tokens, la lista de errores y la tabla de lexemas. La interfaz puede imprimir tokens en el formato solicitado o serializar el resultado completo en JSON. Como cada carácter consumido hace avanzar el índice y la cantidad de operadores es fija, el costo del recorrido crece linealmente con el tamaño de la entrada. Los errores consumen el fragmento problemático y el análisis continúa cuando existe un límite de recuperación identificable.

7.2 Formato de salida

El siguiente fragmento proviene de `corpus/programa_valido.pl`. El quinto campo corresponde al índice de la tabla de lexemas cuando la categoría lo utiliza.

```
<ATOMO, 'padre', 2, 1, 0>
<PARENTESIS_IZQ, '(', 2, 6>
<ATOMO, 'juan', 2, 7, 1>
<COMA, ',', 2, 11>
<ATOMO, 'ana', 2, 13, 2>
<PARENTESIS_DER, ')', 2, 16>
<PUNTO, '.', 2, 17>
```

7.3 Tabla de lexemas

La tabla usa diccionarios que conservan un índice por primera aparición. Los átomos no citados y citados comparten una tabla; las variables poseen otra; y los enteros, reales y cadenas se registran como pares (**tipo**, **lexema**), lo que evita mezclar valores escritos de forma distinta. Se conserva la escritura original: **juan** y **'juan'** tienen entradas distintas, y no se interpretan los escapes ni se convierten números a valores de Python. Los índices son locales a cada tabla y comienzan en cero; el tipo del token identifica cuál debe consultarse. La variable anónima no se interna porque cada aparición representa una variable nueva y no debe compartir identidad.

Tabla 13: Ejemplo de internación sin duplicados.

Tabla	Ejemplo	Regla
Átomos	padre → 0; juan → 1; ana → 2	El mismo lexema reutiliza el índice.
Variables	X → 0; Z → 1; Y → 2	_ se excluye.
Literales	(ENTERO, 52) → 0 y (CADENA, "Ana Perez") → 1	El tipo y la escritura forman la clave.

8 Tratamiento de errores léxicos

Tabla 14: Errores, mensajes y estrategia de recuperación.

Error	Ejemplo	Mensaje	Recuperación
Átomo citado sin cierre	'juan	1:1 Átomo citado sin cierre	Consume hasta salto de línea o EOF y continúa.
Cadena sin cierre	"hola	1:1 Cadena sin cierre	Consume hasta salto de línea o EOF y continúa.
Comentario sin cierre	/* texto	1:1 Comentario de bloque sin cierre	Consume hasta EOF; no existe un límite seguro posterior.
Carácter no admitido	@	1:1 Carácter no admitido	Consume un carácter y continúa.
Número con sufijo	12abc	1:1 Número mal formado	Consume el fragmento ASCII; conserva el punto que no preceda a un dígito.
Exponente incompleto	2e	1:1 Número mal formado	Consume el fragmento numérico completo.
Puntos repetidos	3.14.15	1:1 Número mal formado	Consume el fragmento numérico completo.
Escape no admitido	"x\q"	1:1 Secuencia de escape no admitida	Consume hasta la comilla de cierre.

9 Plan de pruebas y resultados

Los resultados se generaron el 2026-09-22 con Python 3.12.0 mediante `scripts/validate.py`. La ejecución automatizada, reproducida en el anexo, completó 75 pruebas con estado OK. Se conservan 28 pruebas válidas y 12 pruebas de entradas inválidas, posiciones o recuperación del corpus original; se añaden 25 pruebas de regresión del lexer y 10 pruebas de la interfaz. El archivo válido produjo 94 tokens y 0 errores; el archivo con errores produjo 30 tokens, 6 diagnósticos y código de salida 1. Las cifras se incorporan desde `resultados.tex`, generado a partir de la ejecución, y el registro detallado queda en `docs/validation.json`.

Tabla 15: Selección representativa del corpus automatizado.

ID	Entrada	Objetivo	Esperado	Obtenido	Estado
P01	padre	Átomo simple	ATOMO	ATOMO	OK
P03	'Juan Perez'	Átomo citado	ATOMO_ CITADO	ATOMO_ CITADO	OK
P07	-	Anónima sin índice	VARIABLE_ ANONIMA	VARIABLE_ ANONIMA	OK
P10	2e-3	Real con exponente	REAL	REAL	OK
P16	\==	Máxima coincidencia	\==	\==	OK
P24	is island	Límite de palabra	OP / ATOMO	OP / ATOMO	OK
P27	padre(X) repetido	Sin duplicados	Índices 0 y 0	Índices 0 y 0	OK
P29	'juan	Cierre faltante	Error	Error recuperado	OK
P31	/* texto	Comentario abierto	Error	Error hasta EOF	OK
P33	12abc	Número mal formado	Error	Error recuperado	OK
P35	3.14.15	Dos puntos	Error	Error recuperado	OK
P38	a.\n @	Ubicación	2:3	2:3	OK
P39	1e+	Exponente incompleto	Fragmento 1e+	Fragmento 1e+	OK
P40	12abc, ok.	Recuperación numérica	, ok .	, ok .	OK

9.1 Análisis de resultados

El registro JSON incluye huellas SHA-256 de los archivos de código, pruebas, corpus y del propio validador. Antes de calcularlas se normalizan los finales CRLF a LF para comparar copias entre plataformas. El validador comprueba que estas entradas no hayan cambiado durante la ejecución. Las huellas permiten identificar el contenido probado y detectar evidencia desactualizada; no acreditan autoría ni sustituyen las contribuciones registradas en Git.

La cobertura comprueba cada familia léxica, los límites entre palabras y operadores, la deduplicación de lexemas y la actualización de líneas y columnas. Los casos P16 y P24 muestran que la prioridad no se resuelve solo por el orden de las alternativas: primero se consume el lexema completo y después se clasifica. Los casos P39 y P40 verifican que un exponente incompleto se reporte como un solo fragmento y que el analizador retome el reconocimiento en el delimitador siguiente.

El archivo con errores confirma una recuperación parcial. Después de @, 12abc, una cadena abierta al final de línea, 3.14.15 y un escape inválido, el lexer vuelve a reconocer tokens posteriores. En cambio, un comentario de bloque abierto consume hasta el final del archivo porque no existe una marca léxica segura que indique dónde habría terminado.

Las regresiones verifican además saltos LF, CRLF y CR, preservación del punto de cláusula después de

números mal formados y fragmentos de literales que terminan en una barra invertida. La interfaz se comprueba ejecutando procesos reales: formato solicitado en el enunciado, referencias a la tabla de lexemas, recuperación en el corpus inválido, BOM UTF-8, texto Unicode incluso con salida redirigida, archivo vacío, ayuda, archivo inexistente y codificación inválida. Un error de lectura genera un diagnóstico de uso y código 2 sin una traza interna de Python.

La principal limitación está en el alcance: el programa usa identificadores ASCII y no intenta reproducir toda la sintaxis de SWI-Prolog. Tampoco decide si una secuencia de tokens forma una cláusula válida. En consecuencia, los resultados demuestran conformidad con el subconjunto definido en este informe, no compatibilidad total con ISO Prolog.

10 Conclusiones

El analizador cumple el objetivo planteado para el subconjunto definido: reconoce las categorías obligatorias, conserva la posición inicial de cada token, registra átomos, variables y literales sin duplicados innecesarios, y comunica los errores junto con el fragmento que los produce. Esta conclusión se apoya en 75 pruebas automatizadas finalizadas sin fallos y en dos ejecuciones completas. El programa válido produjo 94 tokens sin diagnósticos; el programa inválido permitió recuperar 30 tokens y reportó 6 errores.

La máxima coincidencia resultó decisiva en los operadores con prefijos compartidos. Sin ella, `\==` podría dividirse como `\=` y `=`, mientras que `=..` podría reducirse prematuramente a `=`. La clasificación posterior de identificadores resolvió un problema distinto: `is` se reconoce como operador aritmético, pero `island` permanece como átomo. También se optó por tratar el signo como operador independiente, decisión que mantiene la etapa léxica separada de la interpretación sintáctica de una expresión.

El alcance sigue siendo limitado de forma deliberada. Se admiten identificadores ASCII, un conjunto acotado de escapes y números decimales, pero no todas las extensiones de SWI-Prolog. Un comentario de bloque abierto consume hasta el final del archivo porque no existe un cierre seguro desde el cual reanudar el recorrido. Como trabajo posterior, la solución podría incorporar identificadores Unicode, otras formas numéricas y la generación automática de tablas de transición a partir de una especificación única. Esas ampliaciones tendrían que acompañarse de nuevas pruebas y de la actualización del modelo formal para conservar la coherencia alcanzada.

Referencias

- [1] Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2007). *Compilers: Principles, techniques, and tools* (2.^a ed.). Pearson.
- [2] Hopcroft, J. E., Motwani, R., y Ullman, J. D. (2007). *Introduction to automata theory, languages, and computation* (3.^a ed.). Pearson.
- [3] Python Software Foundation. (2026). *re — Regular expression operations*. Documentación de Python 3. Recuperado el 22 de septiembre de 2026 de <https://docs.python.org/3/library/re.html>
- [4] SWI-Prolog. (s. f.). *The SWI-Prolog syntax*. Recuperado el 22 de septiembre de 2026 de <https://www.swi-prolog.org/pldoc/man?section=syntax>
- [5] SWI-Prolog. (s. f.). *Operators*. Recuperado el 22 de septiembre de 2026 de <https://www.swi-prolog.org/pldoc/man?section=operators>

11 Anexos

Tabla 16: Recursos asociados al informe.

Recurso	Ubicación o enlace
Repositorio Git	https://github.com/Frante23/prolog-lexer-info1148
Código principal	src/lexer.py y src/cli.py.
Pruebas	tests/test_lexer.py, tests/test_lexer_regressions.py y tests/test_cli.py.
Corpus	corpus/programa_valido.pl y corpus/programa_con_errores.pl.
Evidencia	docs/validation.json y informe/resultados.tex, generados por scripts/validate.py.
Compilación	scripts/build_report.py; instrucciones locales y Overleaf en informe/README.md.

11.1 Evidencia de ejecución

La siguiente secuencia reproduce la validación y genera el PDF actualizado. Las cifras se obtienen de la ejecución de las pruebas y del corpus; la duración depende del equipo.

```
python scripts/validate.py
python scripts/build_report.py
python -m src.cli corpus/programa_valido.pl --json
python -m src.cli corpus/programa_con_errores.pl --json
```

Tabla 17: Resultados de la validación con Python 3.12.0.

Ejecución	Pruebas o tokens	Fallos o errores	Salida
Suite automatizada	75	0	0
Archivo válido	94	0	0
Archivo con errores	30	6	1

El informe se puede compilar con pdfLaTeX, XeLaTeX o Tectonic. Los dos primeros motores realizan al menos dos pasadas; Tectonic gestiona sus propias repeticiones. Se comprueba que no queden referencias sin resolver. Se utiliza Latin Modern sin depender del registro de fuentes del sistema ni de Perl. La opción `--publish` valida las pruebas antes de compilar y solo actualiza `informe/main.pdf` si ambos pasos terminan correctamente. La copia para la plataforma es `informe/Tarea_JefeGrupo_JoseJimenez.pdf`.

11.2 Registro de contribuciones

La Tabla 18 cita commits existentes del repositorio y su autor registrado. La revisión parte de `4e40d48` y conserva el historial previo, cuyos commits figuran bajo la identidad `Frante23`. Las correcciones posteriores usan la identidad ya configurada, `deimon005`. La revisión de entrega incorpora además contribuciones bajo la identidad `Pantom21`, confirmada por Jose Jimenez para este trabajo. Estos datos registran autoría Git; no permiten atribuir a otros integrantes actividades que no constan bajo su identidad. Cada integrante debe comprender la solución y registrar sus propias contribuciones, según el enunciado.

Tabla 18: Selección de contribuciones verificables en el historial Git.

Autor Git	Actividad registrada	Commit
Frante23	Implementación inicial del núcleo y la interfaz.	5ab4aa9, ba6f951
Frante23	Pruebas iniciales y fuentes del informe.	c4b3162, 7dbb0f5
deimon005	CLI UTF-8 y validación de archivos completos.	1b18f60
deimon005	Posiciones, recuperación y pruebas de regresión.	8bca7de
deimon005	Evidencia reproducible para el informe.	239d277
deimon005	Compilación portable con ambos motores.	6180328
Pantom21	Pruebas de límites, citas y comentarios.	0ec4058
Pantom21	Huellas del código y corpus validados.	c2e87e9
Pantom21	Trazas formales de aceptación y recuperación.	cf0d3bc
Pantom21	Compilación portable y publicación tras validación.	dd9c508

El historial completo se consulta con `git log --oneline` o desde GitHub Desktop. El jefe de grupo designado es Jose Jimenez; por ello, la copia para la plataforma se denomina `Tarea_JefeGrupo_JoseJimenez.pdf`. El commit que incorpora este PDF se consulta en el historial del archivo; no se incluye su propio hash en el documento para evitar una referencia circular.